Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

По дисциплине: Контейнеризация и оркестрация приложений

Тема работы: Персистентность и изоляция: добавляем БД и настраиваем сети

Обучающийся: Ямалиева Альбина Азатовна

Номер ИСУ: 472706

Факультет: Факультет прикладной информатики

Группа: К3320

Направление подготовки: 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа: Программирование в инфокоммуникационных системах Обучающийся

Обучающийся _____ Ямалиева А. А.

дата, подпись

Руководитель _____ Аминов Н. С.

дата, подпись

Санкт-Петербург

2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Ход выполнения **4**

1.1 Создание именованного тома и запуск PostgreSQL 4

1.2 Проверка персистентности данных 6

1.3 Подключение конфигурации через bind mount 7

1.4 Пользовательская сеть Docker 9

1.5 Проверка сетевой изоляции 10

1.6 Финальная проверка инфраструктуры 11

ЗАКЛЮЧЕНИЕ **13**

ВВЕДЕНИЕ

Цель лабораторной работы — подключить к приложению PostgreSQL, изучить способы хранения данных вне жизненного цикла контейнера, освоить именованные тома и bind mount, а также настроить пользовательскую сеть Docker для изолированного взаимодействия контейнеров.

Таблица 1 — Персональные параметры

Параметр	Значение
Номер ИСУ	472706
Персональный порт PostgreSQL	5436
Персональный порт приложения-ассистента	8006
Персональный префикс	yamalieva
Секретный ключ	secret06yamalieva
Имя базы данных	assistant_06

Результат работы размещён в ветке `lab-3` репозитория: https://github.com/qyalim/course-containerization-orchestration-application/tree/lab-3.

1 Ход выполнения

1.1 Создание именованного тома и запуск PostgreSQL

Для постоянного хранения данных PostgreSQL был создан именованный том `yamalieva-pgdata`:

```
docker volume create yamalieva-pgdata

docker volume ls | grep yamalieva-pgdata
docker volume inspect yamalieva-pgdata
```

Именованный volume существует независимо от контейнера и позволяет сохранить файлы базы данных после удаления и повторного создания контейнера.

PostgreSQL был запущен с персональными параметрами и подключением созданного тома к каталогу данных СУБД:

```
docker run -d \
  --name yamalieva-postgres \
  -e POSTGRES_USER=user_06 \
  -e POSTGRES_PASSWORD=pgpass_06 \
  -e POSTGRES_DB=assistant_06 \
  -v yamalieva-pgdata:/var/lib/postgresql/data \
  -p 5436:5432 \
  postgres:15
```

Внутри контейнера PostgreSQL использует стандартный порт 5432, а на хосте он доступен через персональный порт 5436. На рисунке 1 показан успешный запуск контейнера и проверка его состояния: контейнер `yamalieva-postgres` находится в состоянии `Up`, а порт хоста 5436 перенаправлен на порт 5432 контейнера.

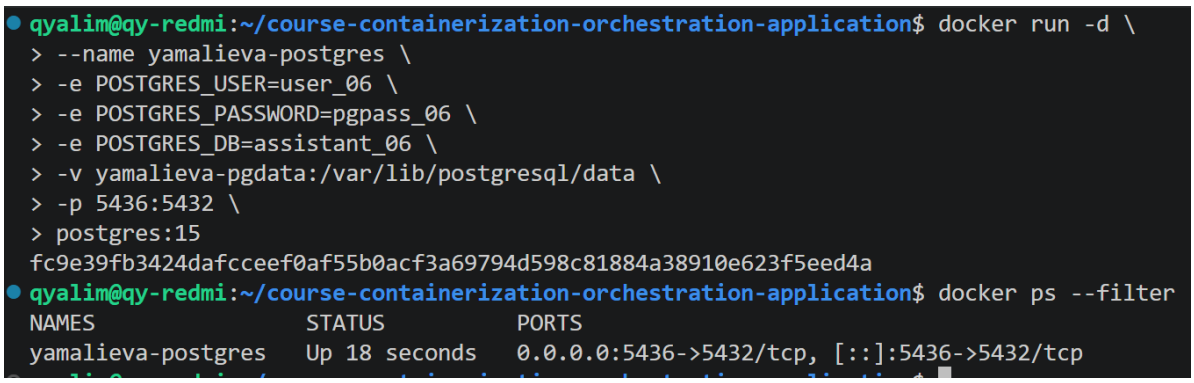


Рисунок 1 — Запуск PostgreSQL с именованным volume и проверка состояния контейнера

После этого был собран образ приложения версии `v3`:

```
1 docker build -t yamalieva-assistant:v3 .
```

Приложение было запущено с переменной `DATABASE_URL`, содержащей параметры подключения к PostgreSQL через адрес хоста `host.docker.internal`:

```
1 docker run -d \
2   --name yamalieva-assistant \
3   -p 8006:8006 \
4   -e APP_OWNER=yamalieva \
5   -e APP_SECRET=secret06yamalieva \
6   --add-host=host.docker.internal:host-gateway \
7   -e DATABASE_URL="postgresql://user_06:pgpass_06@host.docker.intern
    ↪  al:5436/assistant_06" \
8   yamalieva-assistant:v3
```

После запуска были проверены главная страница, состояние приложения, создание заметки и получение списка заметок:

```
1 curl http://localhost:8006/
2 curl http://localhost:8006/health
3 curl -X POST http://localhost:8006/notes \
4   -H "Content-Type: application/json" \
5   -d '{"text": "ЛР3: данные в БД"}'
6 curl http://localhost:8006/notes
```

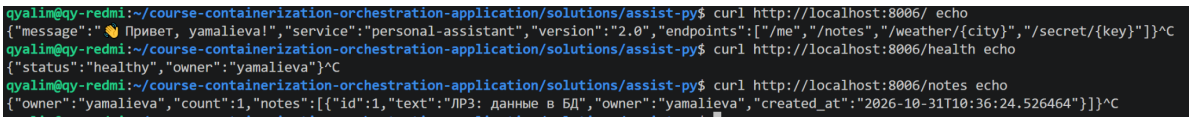


Рисунок 2 — Проверка приложения и работы с PostgreSQL

На рисунке 2 отображены ответы главного эндпоинта, проверки состояния `/health` и запроса списка заметок. Приложение возвращает владельца `yamalieva` и статус `healthy`; в ответе `/notes` присутствует созданная ранее заметка «ЛР3: данные в БД». Это подтверждает работоспособность приложения и успешное сохранение данных в PostgreSQL.

1.2 Проверка персистентности данных

Для проверки персистентности контейнер приложения был остановлен и удалён:

```
1 docker stop yamalieva-assistant
2 docker rm yamalieva-assistant
```

После этого контейнер был создан заново с теми же параметрами подключения к базе данных. Повторный запрос к `/notes` показал, что ранее созданная заметка сохранилась. Это подтверждает, что данные хранятся в PostgreSQL, а не в файловой системе контейнера приложения.

```
1 curl http://localhost:8006/notes
```

Для фиксации результата был выполнен артефакт персистентности с выводом времени, имени пользователя, хоста и списка заметок:

```
1 echo "========================================" && \
2 echo "Тест персистентности: $(date '+%H:%M:%S')" && \
3 echo "$(whoami)@$(hostname)" && \
4 echo "========================================" && \
5 curl -s http://localhost:8006/notes | python3 -m json.tool
```

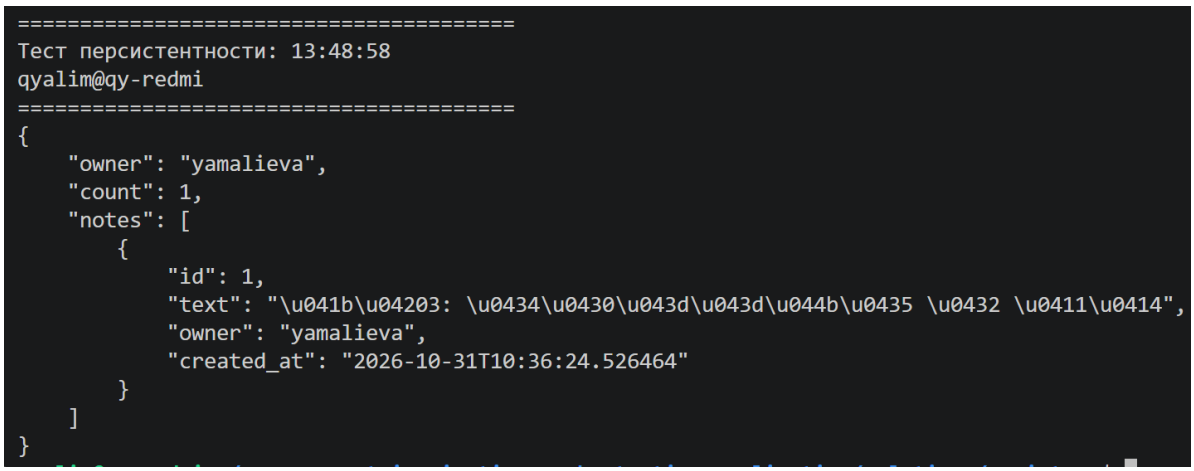


Рисунок 3 — Артефакт проверки персистентности

На рисунке 3 видны время выполнения, логин и имя хоста, а также список заметок после пересоздания контейнера приложения. Запись с идентификатором 1 сохранилась, поэтому удаление контейнера ассистента не привело к потере данных. Персистентность обеспечивается хранением данных в PostgreSQL и именованном volume.

1.3 Подключение конфигурации через bind mount

В каталоге `config` был подготовлен файл `app.conf`:

```
# Конфигурация персонального ассистента
max_notes_per_user=100
enable_weather_api=true
maintenance_mode=true
owner=yamalieva
```

Перед запуском нового контейнера старый контейнер приложения был остановлен, так как порт 8006 может быть занят только одним контейнером одновременно.

Контейнер с конфигурацией был запущен с bind mount:

```
docker run -d \
  --name yamalieva-assistant-config \
  -p 8006:8006 \
  -e APP_OWNER=yamalieva \
  -e APP_SECRET=secret06yamalieva \
  --add-host=host.docker.internal:host-gateway \
```

```
  -e DATABASE_URL="postgresql://user_06:pgpass_06@host.docker.internal:5436/assistant_06" \
  -v $(pwd)/config:/app/config:ro \
  yamalieva-assistant:v3
```

В записи `-v $(pwd)/config:/app/config:ro` первая часть задаёт каталог на хосте, вторая — путь внутри контейнера, а суффикс `:ro` подключает каталог в режиме только для чтения.

Доступ к конфигурации внутри контейнера проверялся командами:

```
docker exec -it yamalieva-assistant-config sh
cat /app/config/app.conf
exit
```

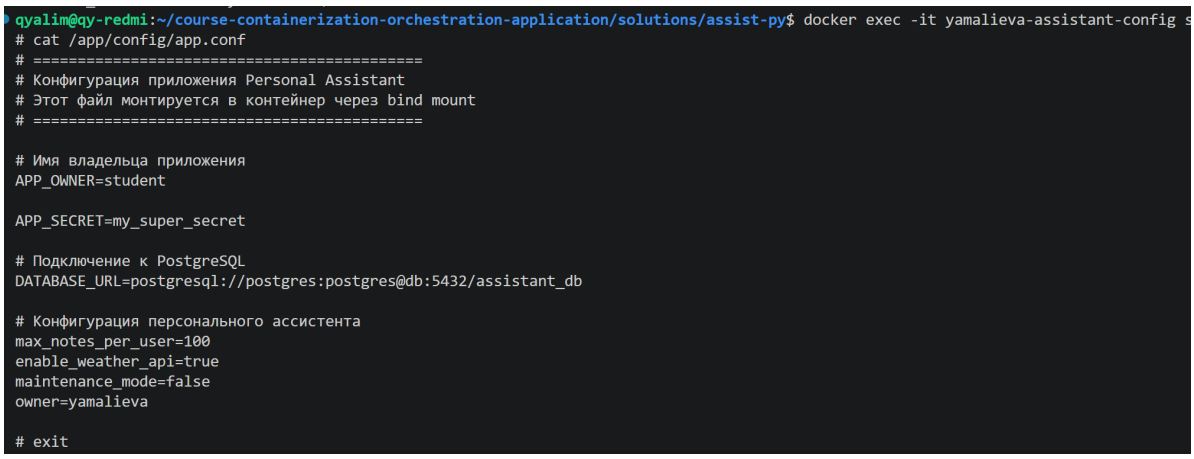


Рисунок 4 — Файл конфигурации внутри контейнера

На рисунке 4 показано содержимое `/app/config/app.conf`, прочитанное непосредственно внутри контейнера. В конфигурации присутствует персональное значение `owner=yamalieva`; это подтверждает корректное подключение каталога `config` через bind mount.

В ходе микро-челленджа значение `maintenance_mode` было изменено с `false` на `true` непосредственно на хосте. После перезапуска контейнера изменённый файл оказался доступен внутри контейнера без пересборки образа. Это демонстрирует отличие bind mount от копирования файла в образ на этапе `docker build`.

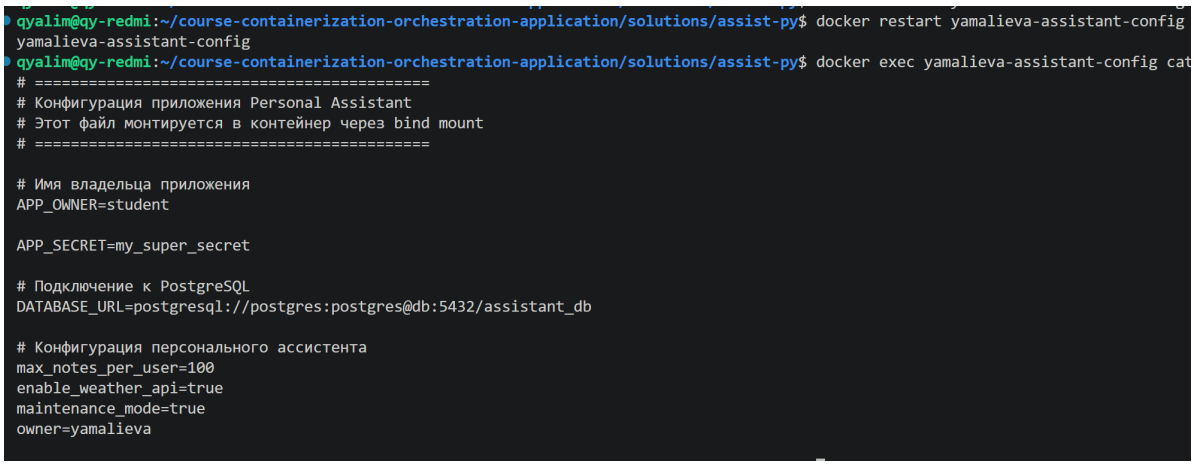


Рисунок 5 — Изменение конфигурации через bind mount без пересборки образа

На рисунке 5 после команды `docker restart` значение `maintenance_mode=tr` видно внутри контейнера. Следовательно, изменение файла на хосте сразу отражается в подключённом каталоге контейнера, а повторная сборка Docker-образа не требуется.

1.4 Пользовательская сеть Docker

Для изоляции сервисов была создана пользовательская bridge-сеть:

```
1 docker network create yamalieva-backend
```

Старые контейнеры PostgreSQL и приложения были остановлены и удалены, после чего PostgreSQL был запущен в сети `yamalieva-backend`:

```
1 docker run -d \
2   --name yamalieva-postgres \
3   --network yamalieva-backend \
4   -e POSTGRES_USER=user_06 \
5   -e POSTGRES_PASSWORD=pgpass_06 \
6   -e POSTGRES_DB=assistant_06 \
7   -v yamalieva-pgdata:/var/lib/postgresql/data \
8   postgres:15
```

В финальной сетевой конфигурации порт PostgreSQL не публикуется на хост, так как приложение обращается к БД напрямую внутри пользовательской сети.

Приложение было запущено в той же сети:

```
docker run -d \
  --name yamalieva-assistant \
  --network yamalieva-backend \
  -p 8006:8006 \
  -e APP_OWNER=yamalieva \
  -e APP_SECRET=secret06yamalieva \
  -e DATABASE_URL="postgresql://user_06:pgpass_06@yamalieva-postgres
  ↪  :5432/assistant_06" \
  -v $(pwd)/config:/app/config:ro \
  yamalieva-assistant:v3
```

Ключевое отличие заключается в том, что в `DATABASE_URL` используется имя контейнера `yamalieva-postgres`. В пользовательской сети Docker встроенный DNS разрешает имена контейнеров в их внутренние IP-адреса.

Работоспособность соединения была проверена запросом к `/health`, после чего была создана дополнительная заметка `«ЛР3: сеть работает!»`.

```
curl http://localhost:8006/health
curl -X POST http://localhost:8006/notes \
  -H "Content-Type: application/json" \
  -d '{"text": "ЛР3: сеть работает!"}'
curl http://localhost:8006/notes
```



Рисунок 6 — Проверка работы приложения и PostgreSQL в пользовательской сети

На рисунке 6 эндпоинт `/health` возвращает состояние `healthy`, а список заметок содержит как исходную запись, так и новую заметку `«ЛР3: сеть работает!»`. Это подтверждает, что после переноса контейнеров в сеть `yamalieva-backend` приложение продолжает успешно обращаться к PostgreSQL по имени контейнера.

1.5 Проверка сетевой изоляции

Для проверки изоляции сначала был запущен временный контейнер Alpine без подключения к сети `yamalieva-backend`:

```
docker run -it --rm alpine:3.18 ping -c 2 yamalieva-postgres
```

Команда завершилась ошибкой `bad address`, поскольку временный контейнер не находился в пользовательской сети и не мог разрешить имя `yamalieva-postgres`.

Затем тот же тест был выполнен с подключением к сети:

```
docker run -it --rm --network yamalieva-backend \
  alpine:3.18 ping -c 2 yamalieva-postgres
```

В данном случае имя контейнера успешно разрешилось и пакеты ping были получены. Таким образом, доступ по внутреннему DNS-имени ограничен участниками одной пользовательской сети.

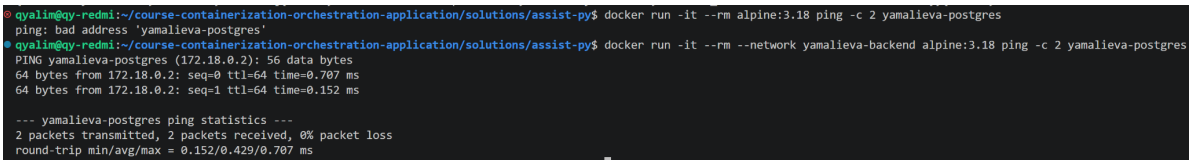


Рисунок 7 — Проверка сетевой изоляции: ping вне сети и внутри сети

На рисунке 7 первая команда запускает Alpine-контейнер вне сети `yamalieva-backend` и завершается сообщением `bad address`, так как имя `yamalieva-postgres` недоступно через Docker DNS. Во второй команде временный контейнер подключён к `yamalieva-backend`; имя PostgreSQL успешно разрешается в адрес `172.18.0.2`, два ICMP-пакета получены без потерь. Это подтверждает сетевую изоляцию и работу встроенного DNS Docker.

1.6 Финальная проверка инфраструктуры

Для итоговой проверки были выведены работающие контейнеры, созданные тома и пользовательские сети:

```
echo "========================================" && \
echo "ЛР3 завершена: $(date '+%d.%m.%Y %H:%M:%S')" && \
echo "Студент: $(whoami)@$(hostname)" && \
echo "========================================" && \
```

```
echo "" && \
echo "Контейнеры:" && \
docker ps --filter "name=yamalieva-" \
  --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}" && \
echo "" && \
echo "Volumes:" && \
docker volume ls --filter "name=yamalieva-" && \
echo "" && \
echo "Сети:" && \
docker network ls --filter "name=yamalieva-"
```

На итоговом выводе присутствуют контейнеры `yamalieva-postgres` и `yamalieva-assistant`, именованный volume `yamalieva-pgdata` и сеть `yamalieva-backend`.

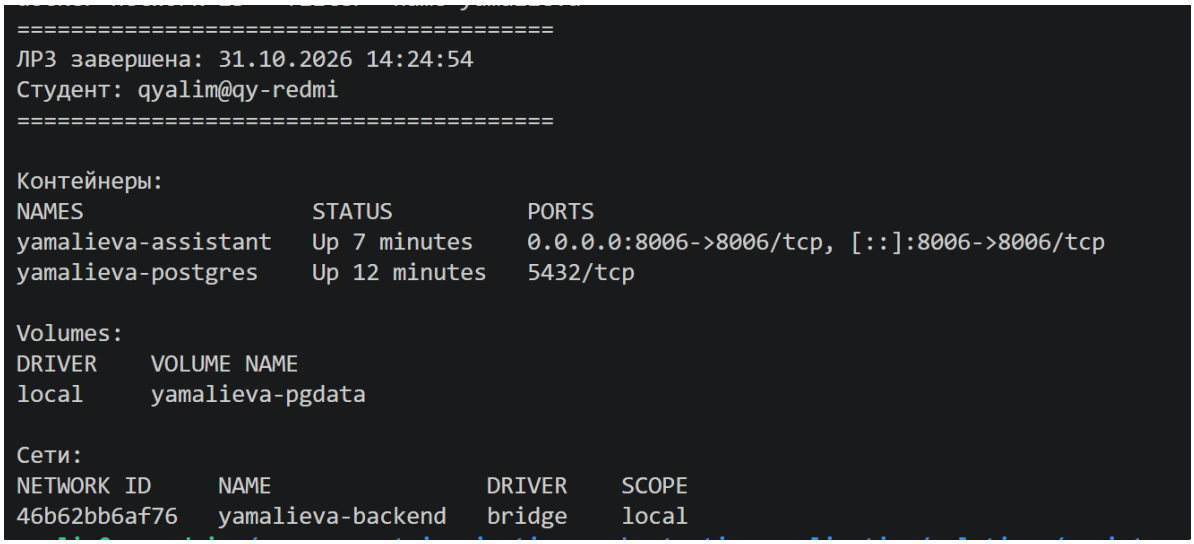


Рисунок 8 — Финальная проверка инфраструктуры

На рисунке 8 одновременно отображаются оба работающих контейнера, именованный volume и пользовательская сеть. `yamalieva-assistant` публикует порт 8006, PostgreSQL доступен только внутри Docker-сети на порту 5432, volume `yamalieva-pgdata` использует локальный драйвер, а сеть `yamalieva-backend` создана с драйвером `bridge`. Таким образом, итоговая инфраструктура соответствует требованиям лабораторной работы.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы приложение было подключено к PostgreSQL, а данные базы размещены в именованном Docker volume. Было подтверждено сохранение заметок после удаления и повторного создания контейнера приложения. Для передачи конфигурации использован bind mount в режиме только для чтения, что позволило изменять конфигурационный файл на хосте без пересборки образа. Также была создана пользовательская сеть Docker, в которой приложение и PostgreSQL взаимодействуют по имени контейнера через встроенный DNS. Проверка с временными контейнерами подтвердила сетевую изоляцию: имя PostgreSQL доступно внутри сети и недоступно контейнеру вне неё. В результате были освоены основные механизмы персистентности и сетевой изоляции контейнеризированных приложений.